B1 Project setup

pyproject.toml, virtual environments, ruff, mypy and folder layout.

B1-L1 One pyproject.toml and a src layout

Logic lives once, in an installable package. Notebooks, jobs and tests import it.

```
reach_model/
├── pyproject.toml
├── src/reach_model/        # importable package
│   ├── __init__.py
│   ├── features.py
│   └── train.py
├── tests/                  # pytest, imports reach_model
├── notebooks/              # exploration; imports reach_model, holds no logic
└── data/                   # small samples only; real data stays out of Git
```

What goes in pyproject.toml?	[project] with dependencies, [dependency-groups] for development tools, and [tool.*] settings for ruff, mypy and pytest.
Install a dependency group	pip install --group dev (pip 25.1 and later).
Why the src layout?	Only the installed package is importable, so tests run against it exactly as jobs do.
Where does shared logic live?	In the package under src/, imported by notebooks, jobs and tests. Never copied.

B1-L2 One virtual environment per project, and python -m pip

One environment per project, created from pyproject.toml, with python -m pip so installing and running use the same interpreter.

```
python -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
python -m pip install --upgrade pip
python -m pip install -e . --group dev
```

Create and use a project environment	python -m venv .venv, activate it, then python -m pip install -e . --group dev.
Why python -m pip?	It installs into the interpreter that python runs. Bare pip may belong to another Python.
Detect a virtual environment in code	sys.prefix != sys.base_prefix.
Two dependencies that cannot coexist	Two pinned environments and two dependency groups, each tested in CI.

B1-L3 ruff and mypy in daily work

ruff and mypy found an unused import, a silent truncation and a crash on zero weight, before anyone wrote a test.

```
m.py:1:1: I001 [*] Import block is un-sorted or un-formatted
m.py:1:8: F401 [*] `os` imported but unused
m.py:9:30: B905 `zip()` without an explicit `strict=` parameter
```

```
m.py:14: error: Unsupported operand types for * ("None" and "int")  [operator]
m.py:17: error: Function is missing a type annotation  [no-untyped-def]
```

Why does ruff flag zip() without strict= (B905)?	zip silently truncates to the shorter input. strict=True raises ValueError instead.
mypy says None * int is unsupported. The fix?	Handle the None case in code. Never # type: ignore it away.
Daily commands before a commit	ruff check --fix ., ruff format ., mypy.
Missing stubs for one library	Install types-<lib> if it exists, otherwise a per-module mypy override. Never global.